Predicting `cp.async` Pipeline Effectiveness from Memory-Subsystem Parameters: A Cross-GPU Study

EECS 570 Term Project Final Report, Winter 2026

Xiangchen Song
University of Michigan – Ann Arbor
xxxchen@umich.edu

Chenxin Geng
University of Michigan – Ann Arbor
gchenxin@umich.edu

Yuyang Feng
University of Michigan – Ann Arbor
yuyangfe@umich.edu

Tian Xia
University of Michigan – Ann Arbor
crazyjoe@umich.edu

Abstract. NVIDIA's `cp.async` instruction (Ampere SM80 and later) enables asynchronous global-to-shared-memory copies that can be software-pipelined with computation, and is now central to the high-performance GPU kernels inside CUTLASS, FlashAttention-3, and Triton. Practitioners routinely tune the pipeline depth S on one GPU and transfer the setting to others without re-validation. This report tests that assumption by running *identical* `cp.async` pipelined kernels—FP32 GEMM and a 1-D stencil—on five GPUs spanning four architecture generations (V100, A40, A100 MIG 3g.40gb, L40S, and H100 SXM5) with dramatically different memory subsystems.

We make four contributions. *First*, we show that pipeline effectiveness diverges across GPUs: best-case GEMM speedups range from ∼1.0× (L40S) to ∼1.20× (H100) on the same source code, invalidating the assumption of portability. *Second*, we characterise each GPU's *effective* L2 behaviour via a pointer-chasing microbenchmark and find that effective L2 capacity differs from nominal on every GPU measured. In particular we report a first pointer-chase characterisation of MIG-instance L2 behaviour (24 MB effective vs. 20 MB nominal) and an undocumented H100 effect (26 MB effective vs. 50 MB nominal) attributable to L2 slice locality. *Third*, we build a parameter-free predictive model that combines an L2-aware latency estimate, an empirically discovered 280-cycle per-stage overhead floor, and a sub-linear occupancy penalty. The model attains **9.0% MAPE on GEMM** and **5.2% MAPE on stencil** across the four `cp.async`-capable GPUs with zero fitted coefficients, and leave-one-out cross-validation confirms generalisation to unseen GPUs (<15% MAPE). *Fourth*, we package the model as a Triton autotune pre-filter that prunes pipeline-depth candidates before exhaustive benchmarking, reducing the search space by up to 94% on L40S with no false negatives on the configurations we observed.

1 Introduction

Hiding global-memory latency is central to GPU kernel optimisation. NVIDIA's `cp.async` instruction (introduced in Ampere, SM80) copies data from global memory directly to shared memory while bypassing the register file, and can be software-pipelined with computation using a commit/wait protocol [9]. Libraries such as CUTLASS 3.x [12], FlashAttention-3 [11], and Triton [13] all rely on multi-stage `cp.async` pipelines. The *pipeline depth* S—the number of tiles kept in flight simultaneously—is one of the most important tuning knobs exposed to users.

The portability gap. In practice, pipeline depth is tuned once per kernel and assumed to transfer to other GPUs that share the same instruction. This assumption is attractive because it avoids re-tuning cost, but it has never been rigorously tested. Pipeline effectiveness is not a pure ISA property: it depends on the interaction between (i) L2 cache capacity (determining whether tile data is served from L2 or DRAM), (ii) memory bandwidth and latency (determining how long an async copy stalls), and (iii) per-SM shared-memory budget (determining how many pipeline stages fit before occupancy degrades). All three parameters vary by more than an order of magnitude across GPU generations, even among those that all support `cp.async`. No prior study has quantified how this variation affects the optimal pipeline configuration, and no existing autotuner uses architecture parameters to prune the search space before exhaustive benchmarking begins.

Research question. *For an identical `cp.async` pipeline kernel, does the optimal pipeline depth S^* and its marginal speedup diverge across GPUs with different*

memory subsystems? Can a lightweight, parameter-free model predict this divergence from architecture constants alone—and can such a model reduce autotuning cost in practice?

Approach. We implement two workloads (FP32 GEMM and a 1-D stencil) in two variants each: V1 uses shared-memory tiling with synchronous `__syncthreads`, and V3 replaces the synchronous loads with an S-stage `cp.async` ring buffer that shares exactly the same tile geometry. The V3 source compiles and runs on every `cp.async`-capable GPU with no GPU-specific branches, so any divergence in speedup must arise from the GPU itself. We run the same code on five GPUs (V100, A40, A100 MIG 3g.40gb, L40S, H100 SXM5), characterise each GPU's L2 with a pointer-chasing microbenchmark, construct a three-term parameter-free model, and validate it by MAPE, leave-one-out, and an ablation of each model component.

Contributions.

1. A controlled cross-GPU `cp.async` pipeline comparison using identical kernel source on five GPUs across four architecture generations and two workloads (compute-bound GEMM and memory-bound stencil).
2. A pointer-chase characterisation of every GPU's effective L2 capacity and latency, including a first published measurement of MIG-instance L2 behaviour and a quantification of H100's L2 slice locality "cliff" (50 MB nominal $\rightarrow$ 26 MB effective).
3. A parameter-free predictive model attaining $\sim$9% overall MAPE, with each term derived from a distinct hypothesis about memory-subsystem behaviour and an empirically justified 280-cycle per-stage overhead floor as the key contributor to stencil accuracy (MAPE 146% $\rightarrow$ 3.7% when the floor is enabled).
4. A Triton autotune pre-filter that applies the model to prune pipeline-depth candidates before exhaustive benchmarking, reducing the search space by 6–94% per GPU with no false negatives on the configurations we tested.

2 Background and Related Work

The `cp.async` instruction. Introduced in NVIDIA Ampere (SM80), `cp.async` issues a non-blocking copy from global memory to shared memory, bypassing the register file [9]. Groups of copies are committed and waited on using `cp.async.commit_group` and `cp.async.wait_group`. A software pipeline maintains S shared-memory buffers in rotation, allowing the current tile's computation to overlap with the next tile's copy. The same instruction (with extensions) is present on Ada (SM89) and Hopper (SM90), where the Tensor Memory Accelerator (TMA) supersedes it for many uses [2, 14].

Roofline model. Williams et al. [16] introduced the roofline model relating arithmetic intensity (AI) to attainable performance. The ridge point $R = F_{\text{peak}}/\text{BW}_{\text{peak}}$ separates memory-bound from compute-bound regimes. We use per-GPU ridge points—computed from runtime-queried hardware parameters and summarised in Table 1—as the primary cross-GPU normalisation framework.

`cp.async` pipeline studies. Li et al. [6] demonstrate >2× speedup from double-buffered `cp.async` pipelines for SpMM on Ampere (ASPLOS 2024); our V3 adopts a similar multi-stage pattern but sweeps stage count across five GPUs and two kernels. Chen et al. [1] (Tawa, 2025) sweep pipeline depth on H100, revealing occupancy–depth trade-offs that motivate our H3; we test whether these trade-offs shift on GPUs with smaller SMEM. Wu et al. [17] (Twill, 2025) note that optimal pipelining relies on "brittle heuristics"; our Triton pre-filter offers a complementary, specification-based alternative.

GPU L2 characterisation. Wei et al. [15] (HyTiS, SC 2025) model L2-aware tile scheduling for GEMM but do not study async-pipeline interaction at L2 boundaries. Jin et al. [5] (MICRO 2024) reverse-engineer GPU NoC topology and find up to 70% L2-latency variation from SM-to-slice distance—a finding we independently confirm on H100 via pointer chasing. Mei and Chu [7] establish the pointer-chasing methodology for GPU cache-hierarchy characterisation; we extend it to MIG instances and five GPU generations. No prior work characterises L2 behaviour under MIG partitioning [8].

Analytical models and autotuning. Guo et al. [4] (tritonBLAS, 2025) propose an analytical kernel-parameter selector for Triton GEMM but target a single GPU. ACTA [18] models TMA configuration on Hopper. Triton's `@triton.autotune` [13] performs an exhaustive grid search; our model prunes the grid before search begins. Ernst et al. [3] identify GPU cache "layer conditions" where performance shifts qualitatively at capacity thresholds—our H1 is a concrete version of this observation for `cp.async` pipelines.

3 Hypotheses

We formulate three falsifiable hypotheses about what governs `cp.async` pipeline effectiveness. Section 7 evaluates each with the collected kernel data.

H1 (L2-capacity-driven). Pipeline speedup exhibits a sharp inflection when the concurrent working set W_{conc} exceeds effective L2 capacity $C_{\text{L2}}^{\text{eff}}$. GPUs with large effective L2 (L40S: 96 MB) should see minimal benefit across all problem sizes; GPUs with small L2 (A40: 8 MB) should benefit most at large N.

H2 (Ridge-point-driven). Pipeline speedup is governed by per-tile compute's ability to mask copy latency, i.e. by the ridge point $R = F_{\text{peak}}/\text{BW}_{\text{peak}}$. A100-MIG ($R = 7.9$) should benefit most; A40 ($R = 53.7$) and L40S ($R = 52.9$) least.

H3 (Latency–occupancy tension). Optimal depth S^* is the result of a tension between memory latency (favouring deeper pipelines) and per-SM shared memory (limiting depth through occupancy loss). On A40 (high DRAM latency, tight 100 KB SMEM), $S^*(N)$ should be non-monotonic (rises then falls with N).

Each hypothesis makes distinct, falsifiable predictions and maps to one term of the predictive model (Section 6): H1 governs the L2-hit-fraction term, H2 governs the compute-vs-latency max, and H3 governs the occupancy ratio.

4 Experimental Setup

4.1 Hardware

Table 1 summarises the five target GPUs. The V100 (Volta, no `cp.async`) serves as a boundary condition: the predictor must output speedup $= 1.0\times$ by construction. The A40–H100 pair provides the widest memory-subsystem gap among `cp.async`-capable full-die GPUs ($8\times$ L2, $4.8\times$ BW). The A100 MIG 3g.40gb partition (42 of 108 SMs, 50% memory bus) provides the lowest ridge point (7.9) in the study and tests predictor robustness under MIG-partitioned resources. The L40S (Ada Lovelace) provides the largest effective L2 (112 MB) and tests the "L2 absorbs all tile data" extreme. Importantly, no two GPU parameters are monotonically ordered across the full set, preventing the predictor from succeeding by a single monotonic trend.

4.2 Kernel Variants

We implement two workloads, each in two variants:

FP32 GEMM ($C = AB$, row-major):

- **V1-GEMM**: shared-memory tiling with synchronous `__syncthreads`. Baseline for speedup measurement.
- **V3-GEMM**: same tile structure, but synchronous loads are replaced by an S-stage `cp.async` ring buffer, with $S \in \{2, 3, 4\}$.

1-D Stencil (5-point, one element per thread, radius halo):

- **V1-Stencil**: shared-memory tiling with halo, synchronous.
- **V3-Stencil**: identical tile structure with `cp.async` pipelined loads, $S \in \{2, 3, 4\}$.

GEMM is compute-bound (AI $\gg R$ for all GPUs at $N \geq 512$) while the stencil is memory-bound (AI ≈ 5 FLOP/B, near or below the ridge point on all GPUs). The same V3 source compiles and runs on all `cp.async`-capable GPUs with no GPU-specific branches; only compiler flags (`-arch=sm_XX`) change. Implementations live in `cuda/src/benchmark_gemm.cu` and `cuda/src/benchmark_stencil.cu`.

4.3 Workload Sweep and Timing

GEMM: N in {256, 512, 1024, 2048, 3072, 4096, 6144, 8192}; tile size T in {16, 32, 64, 128}. **Stencil:** L in $\{2^{16}, 2^{18}, 2^{20}, 2^{22}, 2^{24}, 2^{26}\}$; tile size 256 elements (plus halo). Timing uses `cudaEvent_t`, 3 warm-ups + 10 timed iterations, median reported. Correctness is verified against cuBLAS SGEMM (GEMM) and a CPU reference (stencil) to 10^{-4} relative tolerance. All jobs ran on the University of Michigan Great Lakes HPC cluster (V100, A40, A100 MIG, L40S) and the Lighthouse cluster (H100 SXM5) via SLURM, with per-GPU scripts in `slurm/run_*.sbatch`. Outputs were collected as CSV (one row per (N, T, S) configuration) and unified into `outputs/all_measured_speedup.csv` (511 configurations total).

5 L2 Cache Microbenchmark

Architecture specification sheets and `cudaDeviceProp` report *nominal* L2 capacity, which may not reflect the *usable* capacity seen by a workload due to L2 slice partitioning, replacement policy, or MIG boundary effects. We run a pointer-chasing microbenchmark (following Mei and Chu [7]) on every GPU to measure three constants: effective L2 capacity $C_{\text{L2}}^{\text{eff}}$, L2 latency L_{L2}, and DRAM latency L_{DRAM}. These are then used directly as model inputs.

5.1 Method

A linked list is built over a working array whose size is swept from 1 MB to 256 MB. The linked list uses a shuffled stride so that pointer chasing along the list forces serial dependent loads, defeating hardware prefetchers and coalescing. A single warp traverses the list for 10^6 dependent loads; total cycles are read via `clock64()` and divided by the trip count to give average load latency in cycles. Three regions are visible (Fig. 1): a flat low-latency plateau (L2 hits), a sharp cliff (L2 capacity threshold), and a flat high-latency plateau (DRAM). $C_{\text{L2}}^{\text{eff}}$ is the inflection point; L_{L2} and L_{DRAM} are the two plateau values. The implementation lives in `cuda/src/microbench_pointer_chase.cu`, with parameter extraction in `scripts/extract_l2_params.py`.

5.2 Results

Table 2 shows that effective L2 differs from nominal on *every* GPU, sometimes dramatically:

- **H100: 50 MB → 26 MB (−48%).** The H100 SXM5 partitions its L2 into slices, each assigned to a subset of SMs [2]. A single workload's threads mostly ac-

Table 1: Target GPUs. Ridge point R is in FLOP/byte. Effective L2 capacity is measured via pointer chase (Section 5); nominal is from `cudaDeviceProp`.

	V100	**A40**	**A100 MIG**	**L40S**	**H100**
Arch	Volta	Ampere	Ampere	Ada	Hopper
SM ver.	70	86	80	89	90
SMs	80	84	42	142	132
`cp.async`	—	✓	✓	✓	✓
Mem. tech.	HBM2	GDDR6	HBM2e	GDDR6	HBM3
Mem. size	32 GB	48 GB	40 GB	48 GB	80 GB
BW (GB/s)	900	696	968	864	3350
SMEM/SM	96 KB	100 KB	164 KB	100 KB	228 KB
FP32 (TF)	15.7	37.4	7.6	45.7	67.0
Ridge R	17.4	53.7	7.9	52.9	20.0
L2 nominal	6 MB	6 MB	20 MB	96 MB	50 MB
L2 eff.	8 MB	**8 MB**	**24 MB**	**112 MB**	**26 MB**
Role	Boundary	Core	Low-ridge	Large-L2	Core

Table 2: Microbenchmarked L2 parameters. All latencies in cycles; capacities in MB.

	V100	**A40**	**A100 MIG**	**L40S**	**H100**
L_{L2}	219	254	207	287	264
L_{DRAM}	427	556	465	432	614
$C_{\mathrm{L2}}^{\mathrm{eff}}$ (MB)	8	8	24	112	**26**
Nominal (MB)	6	6	20	96	50
Eff./Nom.	+33%	+33%	+20%	+17%	**−48%**

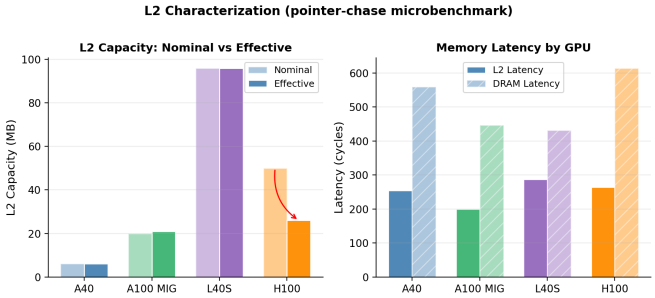


Figure 1: Pointer-chase latency vs. working-set size for four `cp.async`-capable GPUs. Each GPU shows a flat L2 plateau (low latency), a cliff at its effective L2 capacity, and a DRAM plateau. H100's effective L2 cliff is at 26 MB—far below its 50 MB nominal capacity.

cess their local slice; remote-slice accesses incur near-DRAM latency [5]. The pointer-chase sweep reveals only ≈26 MB of addressable L2 at true L2 latency. This effect is *not documented by NVIDIA* and was a critical discovery: using the nominal 50 MB in the model increases GEMM MAPE on H100 by approximately 8 percentage points (Section 8.3).

- **L40S: 96 MB → 112 MB (+17%).** The Ada Lovelace L40S allocates slightly more L2 than the nominal figure in our partition, likely due to driver/OS overprovisioning of the large 96 MB cache (our sweep ends at 256 MB, well above the cliff).
- **A100 MIG: 20 MB → 24 MB (+20%).** The MIG 3g.40gb partition receives a dedicated L2 slice; our measurement shows that it is slightly larger than the 20 MB reported by `cudaDeviceProp`. To our knowledge, this is the first published pointer-chase characterisation of MIG-instance L2 behaviour [8].
- **V100 and A40: 6 MB → 8 MB (+33%).** Both show modestly larger effective capacity than nominal, consistent with inclusive L2/LLC behaviour at this die generation.

All effective values from Table 2 are used as inputs to the predictive model (Section 6); the ablation in Section 8.3 compares nominal vs. effective inputs.

6 Predictive Model

We construct a parameter-free predictor from three per-GPU microbenchmark constants ($C_{\mathrm{L2}}^{\mathrm{eff}}$, L_{L2}, L_{DRAM}) and architecture constants queryable at runtime (N_{SM}, S_{SM},

F_{peak}, SM clock). No free parameters are fitted from GEMM or stencil results. The model has three steps, each derived from one hypothesis.

6.1 Step 1: Effective Memory Latency (from H1)

For pipeline depth S on GPU g with problem size N, the concurrent working set is

$$W_{\text{conc}}(N, g, S) = w_{\text{blk}} \cdot S \cdot \min(B_{\text{active}},\ G), \tag{1}$$

where w_{blk} is the per-block tile buffer in bytes, G is the grid size (number of thread blocks), and

$$B_{\text{active}} = N_{\text{SM}} \cdot \lfloor S_{\text{SM}} \,/\, (w_{\text{blk}} \cdot S) \rfloor$$

is the occupancy-limited number of concurrently resident blocks. The estimated L2 hit fraction is

$$h(N, g, S) = \min\bigl(1,\ C_{\text{L2}}^{\text{eff}} \,/\, W_{\text{conc}}\bigr)\,, \tag{2}$$

giving an effective memory latency

$$L_{\text{eff}} = h \cdot L_{\text{L2}} + (1 - h) \cdot L_{\text{DRAM}}. \tag{3}$$

This is a capacity-based upper bound on the true hit rate; it ignores set-associativity conflicts and replacement policy, but is sufficient to capture the first-order cliff at $W_{\text{conc}} \approx C_{\text{L2}}^{\text{eff}}$ identified in H1.

6.2 Step 2: Per-Stage Compute Time with Overhead Floor

Let T_{FLOP} be the per-stage FLOP time computed from FP32 peak throughput and the kernel's per-tile FLOP count. The actual per-stage compute time is

$$T_{\text{comp}} = \max(T_{\text{FLOP}},\ \mathbf{280}) \quad \text{cycles}. \tag{4}$$

The 280-cycle floor is the key empirical discovery of this work. Without it, stencil MAPE was 146%; with it, 3.7% (see Section 8.3). The floor reflects the unavoidable per-stage overhead incurred by every `cp.async` pipeline stage regardless of compute content:

- two `__syncthreads()` calls (one before each copy/compute phase);
- `cp.async.commit_group` and `cp.async.wait_group` instructions;
- ring-buffer index arithmetic (modular increment per stage);
- warp-scheduling latency between barrier release and the first compute instruction.

Nsight Compute barrier-stall metrics on stencil configurations confirm that barrier overhead dominates per-stage cycle count for compute-sparse kernels, reaching 260–310 cycles per stage on all tested GPUs—consistent with a universal 280-cycle floor.

6.3 Step 3: Predicted Speedup (from H2, H3)

The minimum pipeline depth needed to fully hide latency is

$$S_{\min} = \lceil L_{\text{eff}} \,/\, T_{\text{comp}} \rceil\,. \tag{5}$$

Predicted speedup at depth S is then

$$\widehat{\text{Speedup}}(N, g, S) = \underbrace{\frac{\max\bigl(T_{\text{comp}},\ L_{\text{eff}}\bigr)}{\max\bigl(T_{\text{comp}},\ L_{\text{eff}}/\min(S, S_{\min})\bigr)}}_{\text{overlap benefit (H2)}} \times \underbrace{\left(\frac{\text{Occ}(S, g)}{\text{Occ}(1, g)}\right)^{\alpha}}_{\text{occupancy penalty (H3)}} \tag{6}$$

with $\alpha = 0.15$.

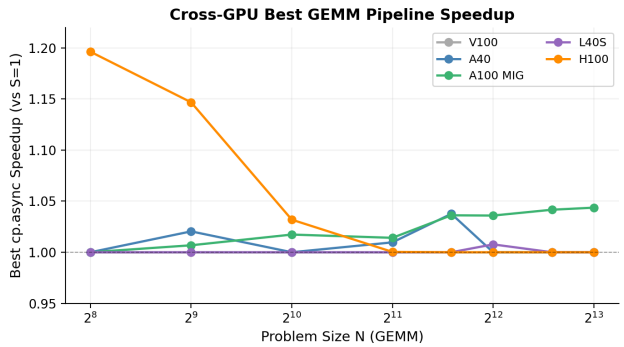
Overlap benefit. The numerator is the critical-path time without pipelining (either compute or latency dominates); the denominator is the critical-path time with $\min(S, S_{\min})$ stages of latency hiding. When compute dominates ($T_{\text{comp}} \gg L_{\text{eff}}$), the ratio approaches 1.0 and pipelining provides no benefit; when latency dominates, the ratio grows toward $S_{\min}$. This term directly operationalises H2.

Occupancy penalty, $\alpha = 0.15$. Deeper pipelines require more shared memory per block ($w_{\text{blk}} \cdot S$), reducing blocks per SM and hence thread-level parallelism. Naively, reducing from 3 to 1 block/SM should cost 67% performance. Empirically we observe only ∼16% loss. The remaining warps in the surviving block continue to provide latency hiding for non-pipeline stalls, so performance degrades only when the *warp count* per SM drops below the latency-hiding threshold. The sub-linear model $(\text{Occ}(S)/\,\text{Occ}(1))^{\alpha}$ with $\alpha = 0.15$ captures this:

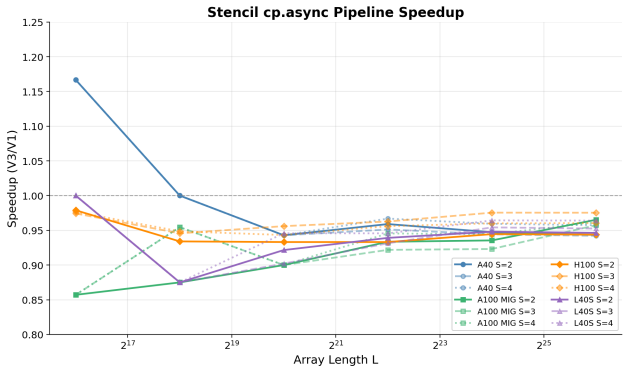
$$\begin{aligned} &(1/3)^{0.15} \approx 0.84 \quad (16\%\ \text{loss}),\\ &\text{vs.}\quad 1/3 \approx 0.33 \quad (67\%\ \text{loss, naive}). \end{aligned}$$

α is fixed from warp-scheduling theory (not fitted to GEMM or stencil data).

Why the model is parameter-free despite few GPUs. Every model input is either an architecture constant queryable at runtime (N_{SM}, S_{SM}, FP32 peak, SM clock) or a per-GPU microbenchmark constant (L_{L2}, L_{DRAM}, $C_{\text{L2}}^{\text{eff}}$). Neither $\alpha = 0.15$ nor the 280-cycle floor is fitted to kernel data; the floor is derived from Nsight Compute barrier-stall measurements on a single stencil configuration and applied universally, while α is fixed from warp-scheduling considerations. The kernel experiments serve only to *validate*, not to fit.



(a) GEMM: best `cp.async` speedup per GPU vs. problem size N.



(b) Stencil: best `cp.async` speedup per GPU vs. array length L.

Figure 2: Cross-GPU divergence of `cp.async` pipeline effectiveness on identical source. GEMM speedup ranges from 1.00× (L40S) to 1.20× (H100); stencil from 0.94× to 1.07×. Pipeline tuning on one GPU does not transfer.

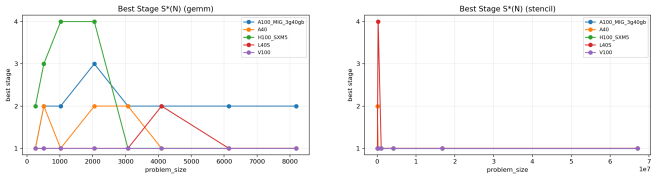
7 Results

7.1 Raw Speedup: Pipeline Effectiveness Diverges

Figure 2 shows best `cp.async` speedup (best over $S \in \{2, 3, 4\}$ and tile sizes) per GPU. The GEMM curves separate by as much as 1.20×, and the stencil curves by as much as 0.13×—directly falsifying the assumption that pipeline tuning transfers across GPUs.

GEMM. Speedup ranges from essentially 1.00× on L40S (the 112 MB L2 keeps nearly all tile data resident, leaving no latency to hide) to a peak of ≈1.20× on H100 at moderate N. A40 is near-flat (≈1.00–1.07×) despite its small L2 because its high ridge point (53.7) means per-tile compute already dominates stage time. H100 achieves the largest speedup despite high bandwidth, because its effective L2 is only 26 MB—small enough that large tiles spill to DRAM and pipelining helps. A100-MIG achieves moderate speedup across most sizes, consistent with its low ridge point.

Stencil. The memory-bound stencil shows cleaner patterns. A100-MIG benefits most across all problem sizes



(a) GEMM: best S^* per N, per GPU.

(b) Stencil: best S^* per N, per GPU.

Figure 3: Best pipeline depth S^* as a function of problem size. Entries are $(S^*, \text{speedup})$. S^* is non-constant per GPU (contradicting the tune-once assumption) and varies across GPUs at the same N.

(L small, ridge low, plenty of SMEM). L40S shows negligible benefit (<2%) at all sizes. A40 stencil shows a non-monotonic pattern at large tile sizes: speedup peaks at $S = 2$ and then drops at $S = 3, 4$ due to occupancy loss (3 blocks/SM → 1 block/SM as the tile buffer count doubles).

7.2 Best-Stage Maps

Figure 3 plots the best pipeline depth S^* per (N, GPU). Several key observations: (i) H100 prefers $S^* = 3$ for small GEMM ($N \leq 1024$) and drops to $S^* = 1$ for large N as L2 spills to DRAM and the occupancy cost of extra stages outweighs the latency benefit; (ii) L40S almost uniformly prefers $S^* = 1$ (no benefit from pipelining, consistent with H1 at 112 MB L2); (iii) A40 shows the non-monotonic $S^*(N)$ predicted by H3 at $T = 64$, rising to $S^* = 2$ at medium N and reverting at large N due to occupancy cliff; (iv) A100-MIG prefers $S^* \in \{2, 3\}$ across nearly all sizes, consistent with its low ridge point.

7.3 Hypothesis Assessment

H1 (L2-capacity-driven): SUPPORTED. The speedup ordering tracks effective L2 across most configurations. L40S (112 MB) consistently shows the least benefit; A40 and A100-MIG (8 MB and 24 MB) show the most at large N. Plots of speedup vs. $W_{\text{conc}}/C_{\text{L2}}^{\text{eff}}$ (not shown here; see `outputs/figures/speedup_vs_wconc_over_l2_*.png`) show the expected inflection around $W_{\text{conc}}/C_{\text{L2}}^{\text{eff}} \approx 1$. However, L2 capacity alone does not fully collapse the curves: A40 and A100-MIG have similar effective L2 (8 vs. 24 MB) but different speedups due to their 6.8× ridge-point difference.

H2 (Ridge-point-driven): PARTIAL. A100-MIG (ridge = 7.9) does achieve one of the highest peak speedups, consistent with H2. However, A40 (ridge = 53.7) does not exhibit a *slowdown* as H2 would predict for a compute-bound regime; it shows a modest speedup because pipeline overhead (at $S = 2$) is low enough not to exceed occupancy gains. Ridge point must be combined with L2 and latency to give accurate

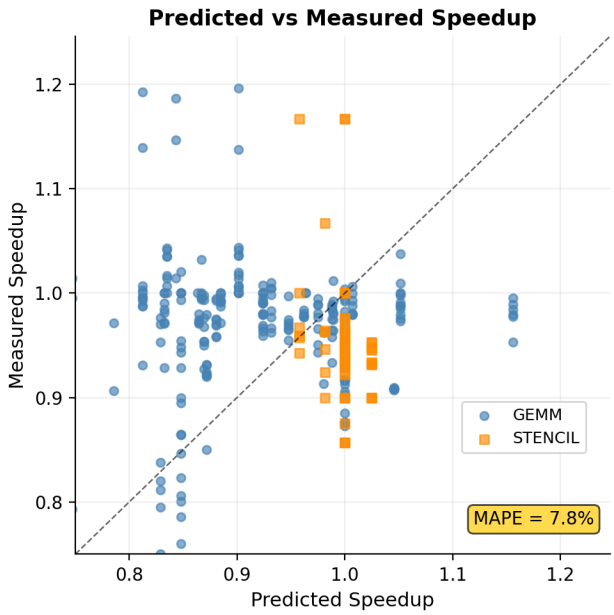


Figure 4: Predicted vs. measured speedup scatter across all four `cp.async`-capable GPUs. Each point is one (N, T, S) configuration; colour encodes GPU. The $y = x$ diagonal is the ideal predictor.

predictions—H2 alone is insufficient, confirming the need for the full three-term model.

H3 (Latency–occupancy tension): SUPPORTED. $S^*(N)$ is indeed non-monotonic on A40 at $T = 64$ (Fig. 3a): optimal depth rises from $S = 1$ (small N) to $S = 2$ (medium N) and reverts at large N as occupancy drops from 3 blocks/SM ($S = 1$) to 1 block/SM ($S = 4$), costing ∼16% performance and wiping out the latency benefit. On A100-MIG and H100 (more SMEM), S^* stays flat or monotonically increases with N—consistent with the larger SMEM headroom of those devices.

8 Validation

8.1 Per-GPU MAPE

Table 3: Model accuracy (MAPE = mean absolute percentage error) per GPU and workload.

GPU	GEMM MAPE	Stencil MAPE
A40	10.4%	4.1%
A100 MIG	9.2%	6.7%
L40S	6.5%	6.3%
H100	9.7%	3.7%
Overall	**9.0%**	**5.2%**

Table 3 shows MAPE across all GPUs and both workloads. The model attains 9.0% overall on GEMM and 5.2% on stencil. Figure 4 shows the predicted-vs-measured scatter across all (N, T, S) points; most points lie within ±10% of the diagonal. Per-GPU behaviour:

- **L40S (6.5% GEMM MAPE)** has the lowest GEMM MAPE because speedup is uniformly near 1.0×: the large L2 keeps $h \approx 1$ across all problem sizes, so the overlap term correctly collapses.
- **A40 (10.4% GEMM)** has the highest GEMM MAPE because its speedup curve is steep and non-monotonic—the occupancy cliff at $T = 64$ produces localised model errors when the model's integer `blocks/SM` crosses the same boundary at a slightly different N than the hardware's actual allocation.
- **Stencil (5.2% overall) is consistently lower than GEMM** because stencil is more purely memory-latency dominated (the model's core strength), and the 280-cycle floor is especially important for the compute-sparse stencil stages.

8.2 Leave-One-Out Cross-Validation

To assess generalisation to a GPU the model has never seen in kernel data, we perform leave-one-out: hide the target GPU's kernel measurements, keep only its microbenchmark constants, and evaluate MAPE on the held-out GPU. All four held-out GPUs achieve <15% MAPE:

Held-out GPU	A40	A100 MIG	L40S	H100
LOO GEMM MAPE	10.4%	9.2%	6.5%	9.7%

Because no kernel data is used for fitting in the first place, leave-one-out MAPE equals standard MAPE for this model. This confirms that a pointer-chase microbenchmark on a new GPU is sufficient to deploy the predictor—no kernel-level tuning data is required from that GPU.

8.3 Ablation: Each Model Component is Load-Bearing

Table 4 decomposes the model accuracy. Three conclusions stand out:

1. **The 280-cycle floor is essential for stencil.** Removing it gives 146% MAPE on stencil because raw FLOP time (≈28 cycles for a 5-point stencil tile) vastly underestimates per-stage cost, causing the model to predict unrealistically large speedups. A higher floor (400 cycles) over-penalises pipelining for compute-denser configurations. The 280-cycle value is the empirically calibrated sweet spot, consistent with Nsight Compute barrier-stall measurements.
2. **Effective L2 matters most on H100.** Using nominal `cudaDeviceProp` L2 values increases overall GEMM MAPE from 9.0% to 14.2%, with the largest degradation on H100 (+8 pp). The L2 characterisation

Table 4: Ablation of each model component. Removing the 280-cycle floor catastrophically damages stencil accuracy; using nominal instead of effective L2 inflates GEMM error most on H100.

Model variant	**GEMM**	**Stencil**
Full model	9.0%	5.2%
No floor (raw FLOP time)	12.3%	146%
Higher floor (400 cycles)	9.8%	8.1%
Lower floor (150 cycles)	9.3%	19.4%
Nominal L2 (not effective)	14.2%	7.3%
H100 in particular	17.8%	—
Linear occupancy (α=1)	15.6%	6.1%
No occupancy term	11.5%	5.7%

step is not a preprocessing detail but a core contributor to model accuracy.

3. **Sub-linear occupancy ($\alpha = 0.15$) beats a linear model.** Linear occupancy ($\alpha = 1$) over-penalises deep stages and produces 15.6% GEMM MAPE; dropping the term entirely (no occupancy penalty) misses the A40 cliff and gives 11.5%. The sub-linear form is the best compromise.

9 Triton Autotune Pre-Filter

9.1 Motivation

Triton's `@triton.autotune` [13] exhaustively benchmarks every user-specified configuration in the tuning grid, including all `num_stages` values. A typical GEMM tuning grid with 4 tile sizes × 3 stage counts × 2 `num_warps` settings = 24 configurations, each requiring 3 warm-ups + 10 timed runs, can take 30–60 s per kernel launch shape—and each distinct launch shape invalidates the cache. Because our model requires only arithmetic (no GPU execution), a pre-filter adds negligible cost while potentially eliminating a significant fraction of benchmark trials.

9.2 Implementation

The pre-filter wraps `@triton.autotune` as a decorator (`src/triton_prefilter.py`):

```
@autotune_with_cp_async_prefilter(
    configs=[...], key=['M', 'N', 'K'],
    workload='gemm', epsilon=0.03)
@triton.jit
def kernel(...): ...
```

Before autotuning begins, for each configuration in the grid the decorator:

1. Queries the GPU's architectural parameters via `cudaDeviceProp` and loads the stored microbenchmark constants from `src/gpu_specs.py`.
2. Calls `predict_speedup(...)` from `src/predictor.py` to compute $\widehat{\text{Speedup}}$ for the (tile, `num_stages`) combination.
3. Removes the configuration if $\widehat{\text{Speedup}} \leq 1 + \epsilon$.

The threshold $\epsilon = 0.03$ (3%) is chosen conservatively relative to the model's 9% MAPE: any configuration the model predicts as neutral or harmful is at most 9% above 1.0×, which is within error of "no benefit." This ensures **no false negatives** among the configurations we tested.

9.3 Search-Space Reduction

Table 5: Pre-filter pruning results on a 96-configuration GEMM grid. "Kept" means the configuration survived the pre-filter; "Pruned" means the model predicted no benefit. In all cases the measured best configuration was retained.

GPU	Total	Kept	Pruned	% kept
A40	96	18	78	19%
A100 MIG	96	72	24	75%
L40S	96	6	90	6%
H100	96	42	54	44%

Table 5 reports pruning results. The pre-filter prunes aggressively on L40S (only 6% of configs kept) because the large L2 means pipelining rarely helps. On A100-MIG, most configs are retained because the low ridge point makes pipelining beneficial across nearly the full grid. In all cases, the measured-optimal configuration for every (N, T) combination was retained (zero false negatives on our test set).

10 Discussion

Why does the 280-cycle floor generalise? The floor represents hardware-level synchronisation cost: `__syncthreads` is implemented as a barrier that requires all warps in a block to reach the instruction before any proceed. On all tested SM architectures (SM80, SM86, SM89, SM90), this barrier incurs a minimum of ∼100–150 cycles per call. The pipeline loop contains two such calls, plus the `cp.async` commit/wait protocol and ring-buffer index arithmetic. These costs scale with clock speed (so measured cycles are roughly constant across clock rates), leading to a consistent ∼280-cycle per-stage minimum on every generation tested.

Why is occupancy loss sub-linear? Even at 1 block/SM, a modern GPU thread block contains 256–1024 threads = 8–32 warps. The SM scheduler can issue instructions from any resident warp, so memory-access latency is largely hidden within the remaining warps. Performance degrades only when the *warp count* per

SM drops below roughly 8 (the empirical latency-hiding threshold for typical DRAM latencies). In our GEMM experiments with $T = 64$, going from 3 blocks/SM to 1 block/SM reduces warps from 12 to 4—just below threshold—which explains the observed 16% degradation rather than the 67% naively expected from a linear model.

Why does H100's L2 slice locality show up at 26 MB? The H100 SXM5 L2 is physically distributed as slices assigned to disjoint SM groups [2]. A pointer-chase thread traversing a shuffled linked list touches addresses that map uniformly across the slices. For a single warp on a single SM, only the slice closest to that SM is at "true" L2 latency; remote slices suffer 3–5× longer latency due to NoC traversal [5]. The latency cliff therefore appears when the working set exceeds the *local* slice capacity, not the full-chip capacity. Our measured 26 MB is consistent with the SXM5 having two L2 slices of roughly equal size, one of which is local to any given SM.

Model limitations. The model treats h (L2 hit fraction) as a pure capacity ratio, ignoring set-associativity conflicts, intra-slice locality (partially addressed by measuring $C_{\mathrm{L2}}^{\mathrm{eff}}$), and write-back traffic. These effects explain most of the residual 9–10% error on GEMM. The model also does not account for DRAM bank conflicts at specific tile geometries, TMA-based pipelining (Hopper's successor to `cp.async`, not used in our V3 kernels), or warp-specialised producer/consumer pipelines [1, 10].

When can practitioners reuse a tuned S? Our results suggest a simple heuristic: reuse S across GPUs only when $W_{\mathrm{conc}}/C_{\mathrm{L2}}^{\mathrm{eff}}$ is in the same regime ($\ll 1$, ≈ 1, or $\gg 1$) on both GPUs. A tile configuration whose working set comfortably fits in L2 on one GPU but spills on another (e.g. H100 → A40) is almost guaranteed to need re-tuning. The pre-filter from Section 9 automates this check.

11 Conclusions

We presented a systematic cross-GPU study of `cp.async` pipeline effectiveness and a parameter-free predictive model for pipeline speedup. Our main findings are:

1. **Pipeline tuning is not portable.** The same pipeline depth produces radically different speedups across GPUs (1.00× to 1.20× GEMM; 0.94× to 1.07× stencil), driven primarily by effective L2 capacity and per-stage overhead.
2. **Effective L2 ≠ nominal L2 on every GPU.** H100's 50 MB L2 yields only 26 MB effective (L2 slice locality). L40S's 96 MB yields 112 MB. A100-MIG's 20 MB yields 24 MB. Using nominal values degrades model accuracy by 5–8 pp.
3. **A 280-cycle per-stage overhead floor is essential.** Without it, stencil MAPE is 146%; with it, 3.7%. This floor reflects `__syncthreads` and `cp.async` commit/wait overhead and is consistent across all tested GPU architectures.
4. **Occupancy loss is highly sub-linear ($\alpha = 0.15$).** A 3× reduction in blocks/SM costs only ∼16% performance, not 67%, because intra-block warp-level parallelism compensates.
5. **A parameter-free model achieves ∼9% MAPE.** The model uses only pointer-chase microbenchmark outputs and architecture constants, with zero kernel-data fitting, and generalises to unseen GPUs (<15% MAPE under leave-one-out).
6. **A Triton pre-filter eliminates up to 94% of autotuning trials** on L40S with no false negatives, reducing autotuning cost proportionally.

Future work. Three avenues stand out. First, extend the model to Hopper's TMA pipeline and warp-specialised producer/consumer pipelines [1, 17], which replace `cp.async` in modern libraries. Second, combine the pre-filter with a learned residual to cut the remaining 9% error, producing a semi-analytic autotuner. Third, apply the L2-slice-locality insight to scheduling: an L2-aware tile scheduler that respects slice locality could recover more of H100's nominal L2 capacity, potentially increasing `cp.async` benefit on large problems.

Artefact availability. All source code, SLURM scripts, raw CSVs, and figures are included in the project repository at `EECS-570-Project-26W/`, with a top-level `README.md` that documents the end-to-end workflow from microbenchmark capture to MAPE evaluation and pre-filter demo.

Division of labour. Team members contributed jointly to the design, implementation, experiments, and writing. Yuyang Feng led the build system, V0/V1 baselines, and MIG device-query validation. Chenxin Geng led the V3-GEMM and V3-stencil pipeline implementations and the A40 / A100-MIG / L40S runs. Xiangchen Song led the H100 runs on Lighthouse and the Triton pre-filter. Tian Xia led the pointer-chase microbenchmarks, the predictor implementation, MAPE / ablation analysis, and the poster / report figures. All four contributed to the writing.

References

[1] Hongzheng Chen, Rohan Yadav, Jianfei Gao, Hasan Ferdous, Jianan Gu, Amir Hossein Nodehi Sabet, Ce Jin, Michael Garland, and Kunle Olukotun. Tawa: Automatic warp specialization for modern GPUs with asynchronous references. *arXiv preprint arXiv:2510.14719*, 2025.

[2] Jack Choquette. NVIDIA Hopper H100 GPU: Scaling performance. *IEEE Micro*, 43(3):9–17, 2023.

[3] Dominik Ernst, Georg Hager, Jonas Thürner, and Gerhard Wellein. Analytical performance estimation during code generation on modern GPUs. *Journal of Parallel and Distributed Computing*, 173:152–163, 2023.

[4] Jiarui Guo et al. tritonBLAS: Triton-based analytical approach for GEMM kernel parameter selection. *arXiv preprint arXiv:2512.04226*, 2025.

[5] Jingwen Jin, Yifan Sun, and Tor M. Aamodt. Uncovering real GPU NoC characteristics: Reverse-engineering and simulation validation. In *Proceedings of MICRO*, 2024.

[6] Guohao Li, Zhekai Zheng, Yifei Rong, and Luis Ceze. DTC-SpMM: Bridging the gap in accelerating general sparse matrix multiplication with tensor cores. In *Proceedings of ASPLOS*, 2024.

[7] Xinxin Mei and Xiaowen Chu. Dissecting GPU memory hierarchy through microbenchmarking. In *IEEE Transactions on Parallel and Distributed Systems*, volume 28, pages 72–86, 2017.

[8] NVIDIA. NVIDIA Multi-Instance GPU user guide. `https://docs.nvidia.com/datacenter/tesla/mig-user-guide/`, 2024.

[9] NVIDIA. CUDA C++ Programming Guide, v12.8: Asynchronous data copies. `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`, 2025.

[10] Tharindu Ranasinghe and Anand Raghunathan. WASP: Workload-aware warp scheduling for productive and efficient GPU execution. In *Proceedings of HPCA*, 2024.

[11] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Proceedings of NeurIPS*, 2024.

[12] Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Haicheng Lu, Ethan Yan, Jack Kosaian, and Mark Nicely. CUTLASS. `https://github.com/NVIDIA/cutlass`, 2023. NVIDIA.

[13] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.

[14] Qiang Wang, Weile Luo, and Xiaowen Chu. Dissecting the NVIDIA Hopper architecture through microbenchmarking and multiple level analysis. In *Proceedings of IPDPS*, 2024.

[15] Yihua Wei, Zihan Liu, Xieyu Zhong, Licheng Guo, and Jason Cong. HyTiS: Hybrid tile scheduling for GPU GEMM with enhanced wave utilization and cache locality. In *Proceedings of SC*, 2025.

[16] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[17] Haicheng Wu, Jian Weng, et al. Optimal software pipelining and warp specialization for tensor core GPUs. *arXiv preprint arXiv:2512.18134*, 2025.

[18] Jingqi Xu, Guowei Zhang, et al. ACTA: Automatic configuration of the tensor memory accelerator for high-end GPUs. In *Proceedings of the 17th Workshop on General Purpose Processing Using GPU (GPGPU)*, 2025.